



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U02_A04_CPE Caso práctico extendido – Prof. Emilio
Ramirez**

Autor: Segura Abad, Mario

Fecha: 22 de Octubre, 2025

ÍNDICE

INTRODUCCIÓN 3

ENUNCIADO.....4

GESTOR DE GASTOS: LISTADO DE GASTOS.....4

DESARROLLO DE LA PRACTICA..... 5

DESCRIPCIÓN.....5

CONTENIDO DEL ARCHIVO.....5

CASO PRÁCTICO EXTENDIDO – LISTADO DE GASTOS..... 5

VALIDACIÓN Y PRUEBAS..... 6

CÓMO VISUALIZARLO7

REQUISITOS: 7

PASOS 7

OBJETIVOS DE LA PRÁCTICA.....8

- MEJORAR LA ESTRUCTURA DEL OBJETO GASTO CON PROPIEDADES AVANZADAS. 8
- IMPLEMENTAR FUNCIONES PARA GESTIONAR UNA LISTA DE GASTOS..... 8
- APLICAR VALIDACIÓN DE FECHAS Y ETIQUETAS. 8
- UTILIZAR DATE() Y toLocaleString() PARA MOSTRAR FECHAS EN FORMATO LOCALIZADO.8
- SUPERAR LOS TESTS PROPUESTOS Y DOCUMENTAR EL CÓDIGO CORRECTAMENTE..... 8
- PRESENTAR EL PROYECTO EN FORMATO PDF JUNTO CON LOS ARCHIVOS FUENTES Y
CAPTURAS DE PANTALLA DE EJECUCIÓN E IMPLEMENTACIÓN. 8

CONCLUSIÓN9

BIBLIOGRAFÍA 10

INTRODUCCIÓN

En el ejercicio “Gestor de gastos: listado de gastos” deberemos realizar un caso práctico que ejemplifica los contenidos vistos y estudiados en clase, del Apartado 4 de la teoría de JavaScript.

ENUNCIADO

Gestor de gastos: listado de gastos

Adjunto, te proponemos un caso práctico que ejemplifica los contenidos.
Ejemplo de implementación:

```
JS gestionPresupuesto.js ×
DWECE_U02_A04_z_CPE_E > js > JS gestionPresupuesto.js > CrearGasto
1 // Autor: Mario Segura Abad
2 // Fecha: 22/10/2025
3
4
5 // Variables globales
6 let presupuesto = 0;
7 let gastos = [];
8 let idGasto = 0;
9
10 // Funciones para Presupuesto
11 function actualizarPresupuesto(valor) {
12     if(typeof valor === "number" && valor >= 0) {
13         presupuesto = valor;
14         return presupuesto;
15     }
16     console.error("Valor inválido.");
17     return -1;
18 }
19
20 function mostrarPresupuesto() {
21     return `Tu presupuesto actual es de ${presupuesto} €`;
22 }
23
24 // Función constructora de gasto
25 function CrearGasto(descripcion, valor = 0, fechaString, ...etiquetas) {
26     if(typeof valor !== "number" || valor < 0) valor = 0;
27
28     let fecha = Date.parse(fechaString);
29     fecha = isNaN(fecha) ? Date.now() : fecha;
30 }
```

[Fecha y Hora - Gestión y hora con el objeto Date\(\)](#)

[Date.prototype.toLocaleString\(\) - Convertir fechas a cadenas en formato localizado \(MDN\)](#)

DESARROLLO DE LA PRACTICA

Práctica DWEK – Unidad 4

Descripción

Este proyecto amplía el desarrollo del gestor de presupuesto personal iniciado en prácticas anteriores.

El objetivo principal ha sido mejorar los objetos Gasto añadiendo propiedades como fecha y etiquetas, y crear funciones para gestionar una lista de gastos.

Se ha trabajado sobre el archivo ya proporcionado y hecha la plantilla `gestionPresupuesto.js`, y se han ejecutado los tests con `npm run test2` (en el Caso práctico anterior fue el `test1`) para verificar el correcto funcionamiento del código.

Además, se ha aplicado la gestión de fechas con el objeto `Date()` y el método `toLocaleString()` para mostrar la información en formato totalmente localizado.

Contenido del archivo

Caso práctico extendido – Listado de gastos

1. Mejora del objeto Gasto

Se ha actualizado y mejorado, dentro de sus posibilidades, la función constructora `CrearGasto` para incluir:

- fecha: almacenada como timestamp.
- etiquetas: array de categorías asociadas.
- Validación de fecha y etiquetas mediante parámetros opcionales.

Ejemplo de implementación:

```
23
24 // Función constructora de gasto
25 function CrearGasto(descripcion, valor = 0, fechaString, ...etiquetas) {
26     if(typeof valor !== "number" || valor < 0) valor = 0;
27
28     let fecha = Date.parse(fechaString);
29     fecha = isNaN(fecha) ? Date.now() : fecha;
30
31     const gasto = {
32         descripcion,
33         valor,
34         fecha,
35         etiquetas: [],
```

También se han añadido métodos al objeto:

- `mostrarGastoCompleto()`: muestra toda la información del gasto en formato localizado.
- `actualizarFecha(fechaString)`: actualiza la fecha si es válida.
- `anyadirEtiquetas(...etiquetas)`: añade etiquetas sin duplicados.
- `borrarEtiquetas(...etiquetas)`: elimina etiquetas existentes en el array.

Ejemplo de implementación:

```

47     },
48     actualizarFecha(nuevaFechaString) {
49       const nuevaFecha = Date.parse(nuevaFechaString);
50       if(!isNaN(nuevaFecha)) {
51         this.fecha = nuevaFecha;
52       }
53     },
54     (parameter) nuevasEtiquetas: any[]
55     anyadirEtiquetas(...nuevasEtiquetas) {
56       nuevasEtiquetas.forEach(etiqueta => {
57         if(!this.etiquetas.includes(etiqueta)) {
58           this.etiquetas.push(etiqueta); // Si no existia la etiqueta en cuestión en el array de etiquetas, se añade al array, ".push(etiqueta)"
59         }
60       });
61     },
62     borrarEtiquetas(...etiquetasAEliminar) {
63       this.etiquetas = this.etiquetas.filter(etiqueta => !etiquetasAEliminar.includes(etiqueta)); // El "filter" conservará aquellas etiquetas que NO estén en "etiquetasAEliminar" y eliminará las que sí lo estén
64     },
65     mostrarGastoCompleto() {
66       const fechaLocal = new Date(this.fecha).toLocaleString();
67       const etiquetasTexto = this.etiquetas.map(etiqueta => ` - ${etiqueta}`).join("\n");
68       return `Gasto correspondiente a ${this.descripcion} con valor ${this.valor} €. \nFecha: ${fechaLocal}\nEtiquetas:\n${etiquetasTexto}`;
69     }
70   };

```

2. Gestión de la lista de gastos

Se han creado variables globales:

- gastos: array que almacena todos los gastos.
- idGasto: contador autoincremental para asignar identificadores únicos.

Ejemplo de implementación:

```

4
5   // Variables globales
6   let presupuesto = 0;
7   let gastos = [];
8   let idGasto = 0;
9

```

Y se han implementado funciones:

- listarGastos(): devuelve el array de gastos.
- anyadirGasto(gasto): añade un gasto al array y asigna un id.
- borrarGasto(id): elimina un gasto por su id.
- calcularTotalGastos(): suma el valor de todos los gastos.
- calcularBalance(): calcula el presupuesto restante.

Ejemplo de implementación:

```

74
75 // Funciones nuevas
76 function listarGastos() {
77   return gastos;
78 }
79
80 function anyadirGasto(gasto) {
81   gasto.id = idGasto;
82   idGasto++;
83   gastos.push(gasto); // "gasto.id = idGasto" se le asigna un identificador único al gasto, si eso se cumple el contador global incrementa y añade el gasto al array de gastos
84 }
85
86 function borrarGasto(id) {
87   gastos = gastos.filter(gasto => gasto.id !== id); // Conserva sólo aquellos gastos cuyo id sea diferente al que se quiere eliminar
88 }
89
90 function calcularTotalGastos() {
91   return gastos.reduce((total, gasto) => total + gasto.valor, 0); // Gracias a ".reduce()" recorre el array y acumula un resultado, el acumulador empieza en 0
92 }
93
94 function calcularBalance() {
95   return presupuesto - calcularTotalGastos();
96 }
97

```

Validación y pruebas

Se han definido todas las funciones requeridas en el fichero para evitar errores de sintaxis. Se han ejecutado los tests (en este caso el_test) con `npm run test2` para comprobar el avance e intentar corregir los máximos errores posibles.

Ejemplo de ejecución:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

mariossegura@MacBook-Air-de-Mario U02_A04_CPE Caso práctico extendido % cd DWEC_U02_A04_z_CPE_E
mariossegura@MacBook-Air-de-Mario DWEC_U02_A04_z_CPE_E % 

mariossegura@MacBook-Air-de-Mario U02_A04_CPE Caso práctico extendido % cd DWEC_U02_A04_z_CPE_E
mariossegura@MacBook-Air-de-Mario DWEC_U02_A04_z_CPE_E % npm run test2

> practica_dwec_gestor_presupuesto@1.0.0 test2
> mocha test/02_testIntroJSII.js

Función listarGastos e inicialización de la variable global gastos
  ✓ Devuelve un array vacío si no hay gastos (valor por defecto)

Función CrearGasto y funcionamiento de objeto gasto
  ✓ CrearGasto: comprobación con fecha y etiquetas (45ms)
    1) Método 'mostrarGastoCompleto' del objeto gasto
  ✓ Método 'actualizarFecha' del objeto gasto
  ✓ Método 'anyadirEtiquetas' del objeto gasto
  ✓ Método 'borrarEtiquetas' del objeto gasto

Función anyadirGasto
  ✓ Añade el gasto que se pasa como parámetro a la variable global 'gastos'

Función borrarGasto
  ✓ Elimina de la variable global 'gastos' el gasto cuyo id se pasa como parámetro

Función calcularTotalGastos
  ✓ Calcula la suma de todos los gastos presentes en la variable global 'gastos'

Función calcularBalance
  ✓ Calcula el balance (presupuesto - gastos totales) a partir de los gastos almacenados en la variable global 'gastos'

9 passing (277ms)
1 failing

1) Función CrearGasto y funcionamiento de objeto gasto
   Método 'mostrarGastoCompleto' del objeto gasto:

AssertionError: expected 'Gasto correspondiente a descripción d...' to equal 'Gasto correspondiente a descripción d...'
+ expected - actual

-Gasto correspondiente a descripción del gasto con valor 23.55 €.
+Gasto correspondiente a descripción del gasto con valor 23.55 €.
Fecha: 10/6/2021, 3:10:00 PM
Etiquetas:
- casa
- supermercado
-- comida
+- comida

```

Cómo visualizarlo

Requisitos:

- Editor de código compatible: en mi caso, Visual Studio Code.
- Node.js: para ejecutar los tests con `npm run test2`.
- Navegador web (opcional): para visualizar los resultados con una interfaz HTML.

Pasos

1. Guardar el archivo `gestionPresupuesto.js` en la carpeta raíz del proyecto.
2. Abrir la terminal dentro de la carpeta raíz, en este caso `DWEC_U02_A04_z_CPE_E`, muy importante este último paso.
3. Ejecutar los tests con: `npm run test2`
4. Verificar que se superan los tests y que las funciones devuelven los resultados esperados.
5. Si se ha creado una interfaz HTML (que no es mi caso), abrirla en el navegador y revisar la consola del navegador (F12).

Objetivos de la práctica

- Mejorar la estructura del objeto Gasto con propiedades avanzadas.
- Implementar funciones para gestionar una lista de gastos.
- Aplicar validación de fechas y etiquetas.
- Utilizar `Date()` y `toLocaleString()` para mostrar fechas en formato localizado.
- Superar los tests propuestos y documentar el código correctamente.
- Presentar el proyecto en formato PDF junto con los archivos fuentes y capturas de pantalla de ejecución e implementación.

CONCLUSIÓN

Esta práctica nos ha permitido aprender y consolidar el uso de objetos y arrays en JavaScript, aplicando técnicas de validación y gestión de datos.

Se ha trabajado con fechas y etiquetas para enriquecer la información de cada gasto, y se ha creado una estructura modular que permite añadir, eliminar y consultar gastos de forma eficiente. Además, se ha reforzado el uso de pruebas automáticas como herramienta de verificación, y se ha aplicado documentación técnica para una entrega que cumpla los requisitos.

La práctica nos ha sido clave para intentar entender cómo construir una librería totalmente funcional y reutilizable con JavaScript.

BIBLIOGRAFÍA

- <https://es.javascript.info/date#creacion>: para entender la gestión de fecha y hora en JavaScript e intentar aplicarlo de la mejor manera posible en el ejercicio.
- https://developer.mozilla.org/es/docs/Web/JavaScript/Reference/Global_Objects/Date/toLocaleString